

KMXTool 設定マニュアル

KMXTool を使って KMX（3D シミュレータ）の設定を行うための手順書です。KMXTool を初めて使う方を対象に、概要から実際の設定例までを順に説明します。

対象：はじめて使う方

所要：初回セットアップ 30分程度

前提知識：不要

目次

- 01 KMX と KMXTool とは
 - 02 準備するもの
 - 03 画面の構成
 - 04 設定の流れ
 - 05 設定手順
 - 06 ユニットタイプ一覧
 - 07 データ通信設定の詳細
 - 08 拡張機能の詳細
 - 09 設定例
 - 10 覚えておきたい約束事
 - 11 よくあるつまずき
 - 12 用語集
 - A 出力ファイル一覧
-

01 KMX と KMXTool とは

KMX (Kinetic Machine eXplorer)

実機の動きを 3D で再現するシミュレータです。Unity で作られています。

- 機械の 3D モデルを画面に表示する
- PLC から受け取った値、またはタイムチャート（時間表）に従って動かす
- ワーク（製品）の流れ、センサ、ロボット、コンベアなどを再現する

KMX 自体は「**設定ファイルを読んで、その通りに動かすだけ**」のソフトです。どの部品をどう動かすかは、すべて設定ファイルで決まります。

KMXTool

その**設定ファイルを作るための専用ツール**です。Windows アプリです。

KMX の設定ファイルは 30 種類以上の JSON ファイルで構成されており、手書きするのは現実的ではありません。KMXTool は画面上で設定を組み立て、必要な JSON ファイル一式をまとめて出力します。

2つの関係



重要

KMXTool と KMX は直接つながっていません。**JSON ファイル**を介してやり取りします。したがって KMXTool で設定を変えただけでは KMX に反映されず、必ず「設定ファイル出力」と、KMX 側での「F5 再読込」が必要です。

02 準備するもの

項目	内容	必須
KMXTool	設定ツール本体	○
KMX	シミュレータ本体（ <code>KMX.exe</code> ）または Unity 開発環境	○
3D モデル	Unity の Prefab。機械の形状データ	○
PLC 接続情報	IP アドレス、ポート番号、デバイス番号	実機接続時

項目	内容	必須
MMS 設定ファイル	PLC のタグ定義 (<code>.xml</code>)	タグ流用時
DXF ファイル	平面レイアウト図	図面利用時

注意

3D モデル (Prefab) は KMXTool では作れません。Unity 側であらかじめ用意しておく必要があります。

03 画面の構成

メニュー

メニュー	項目	ショートカット	内容
ファイル	新規作成	<code>Ctrl+Shift+N</code>	プロジェクトを新規作成
	開く	<code>Ctrl+Shift+O</code>	既存プロジェクト (<code>.json</code>) を開く
	上書き保存	<code>Ctrl+S</code>	プロジェクトを保存
	最近使ったプロジェクト	<code>Ctrl+Shift+1</code> ~ <code>0</code>	履歴から開く
	終了	—	—
プロジェクト	設定ファイル出力	<code>Ctrl+E</code>	KMX 用の JSON を出力 (最重要)
	Prefab 表示	<code>Ctrl+D</code>	Prefab の構造を確認
	ビルド設定	—	KMX のビルド設定
基本設定	—	—	Unity データパスの設定

間違えやすい

「上書き保存」と「設定ファイル出力」は別物です。

- 上書き保存 (`Ctrl+S`) … KMXTool のプロジェクトファイルを保存するだけ
- 設定ファイル出力 (`Ctrl+E`) … KMX が読む JSON を書き出す

保存しただけでは KMX には反映されません。

プロジェクトツリー（画面左側）

設定は木構造で管理されます。上から順に絞り込んでいくイメージです。

KMX Projects

プロジェクト

システム設定

データベース

PostgreSQL

PostgreSQL設定

登録機番

機番

タイムチャート設定

非表示モデル設定

ワーク設定

経路設定

データ通信設定

使用可能タグリスト

データ交換設定

I/Oタイミング設定

内部デバイス

外部ファイルデータ

ユニット構成

ユニット一括設定

ユニット個別設定

ユニット

モデル設定

拡張機能

ワーク操作設定

センサ設定

吸引設定

物体形状設定

機構拡張設定

バケット設定

...

 案件ひとつ分

...

 出力先、MMSファイル

...

 接続先ひとつ分

...

 接続情報・通信方式

...

 機械1台分

...

 サイクル時間など

...

 製品（ワーク）の定義

...

 搬送経路の定義

...

 DXF等

...

 動く部品1つ分

...

 どの3Dモデルか

階層	単位	例
プロジェクト	案件	R0230_新型KTB-100
PostgreSQL	接続先	実機 PLC 1 系統

階層	単位	例
機番	機械 1 台	R0230
ユニット	動く部品 1 つ	シート取出しX軸、コンベア1

ポイント

ユニットが設定の主役です。「動くもの 1 つ = ユニット 1 つ」と考えてください。

04 設定の流れ

- ① 基本設定
- ② プロジェクト作成
- ③ システム設定
- ④ PostgreSQL設定
- ⑤ 機番追加
- ⑥ タイムチャート設定
- ⑦ データ通信設定
- ⑧ ユニット追加
- ⑨ モデル設定
- ⑩ 動作設定
- ⑪ 設定ファイル出力
- ⑫ KMXで確認

実際の作業では ⑧～⑫ を何度も繰り返すことになります。①～③ は初回のみです。

05 設定手順

基本設定（初回のみ）

メニュー「基本設定」を開き、**Unity データパス**を指定します。ここで指定したフォルダが設定ファイルの出力先になります。指定できるのは次の3パターンです。

パターン	指定するフォルダ	実際の出力先
開発環境	Kinetic Machine eXplorer.sln があるフォルダ	...\Assets\StreamingAssets\Datas
実行環境	KMX.exe があるフォルダ	...\KMX_Data\StreamingAssets\Datas
直接指定	StreamingAssets 直下のフォルダ	そのフォルダ

注意

上記のいずれにも当てはまらないフォルダを指定すると、出力時に「**下記パスは不正です**」と表示され、出力されません。迷ったら **KMX.exe があるフォルダ**を指定してください。

この設定は **KMXTool 全体の設定**（プロジェクトごとではありません）です。

プロジェクトを新規作成

「[ファイル] → [新規作成]（**Ctrl+Shift+N**）」でプロジェクト名を入力します。作成後、[ファイル] → [上書き保存]（**Ctrl+S**）で保存先を指定します。プロジェクトファイルは **.json** 形式です。

システム設定

ツリーの「システム設定」を選択します。

項目	内容
保存フォルダパス	2 つめの出力先 。Step 1 と合わせて最大 2 か所へ同時出力できる
MMS 設定ファイルパス	PLC のタグ定義（ .xml ）を登録。タグを一から作らずに済む

保存フォルダパスの判定ルールは Step 1 と同じです。1 か所で足りる場合は空欄で構いません。

PostgreSQL 設定

ツリーの「PostgreSQL」→「PostgreSQL設定」を選択します。ここで **KMX がどこからデータを受け取るか** を決めます。

項目	内容	例
ホスト名	データベースのアドレス	192.168.3.210
ポート番号	—	5432
DB 名	—	kcp_db
ユーザー名／パスワード	—	postgres
サイクル	通信周期 (msec)	10
クライアントモード	受信のみ行う	—
機能	動作モードの選択	下表

機能	内容	使う場面
内部処理	PLC につながらず、KMX 内部だけで動かす	まず動かしてみたいとき
直接通信	PLC と直接通信する	実機接続時
MQTT	MQTT ブローカ経由	—
Redis	Redis 経由	—

おすすめ

初めての場合は「内部処理」から始めてください。PLC がなくてもタイムチャートだけで動きを確認できます。

「直接通信」を選んだ場合は「通信設定」でプロトコル（MC プロトコル、OPC UA など）、IP アドレス、ポート番号を指定します。

機番を追加

「登録機番」を右クリックして機番を追加します。機番は**機械 1 台を識別する ID**です（例：**R0230**）。複数台を 1 つのプロジェクトで扱う場合は機番を複数登録します。

タイムチャート設定（機械全体）

機番配下の「タイムチャート設定」を選択します。機械全体の時間に関する設定です。

項目	内容
1サイクル時間	機械が 1 周する時間（msec）。最も重要
連続動作時間範囲	連続動作とみなす時間範囲（msec）
通信遅れ時間	通信の遅延を模擬する時間（msec）
ON 最低保証時間	信号 ON を保証する最短時間（msec）
サイクルタイプ	サイクルの進み方
加速タイプ	加減速のカーブ
入力タグ／出力タグ	シミュレーション用のタグ
サイクルタグ	サイクル位置を示すタグ
動作テーブルファイル(Excel)	動作テーブルを Excel から取り込む

1 サイクル時間は、後で作る動作テーブルの時間軸の基準になります。最初に決めてください。

データ通信設定

ページ	内容
使用可能タグリスト	使えるタグの一覧。MMS ファイルを登録するところに読み込まれる
データ交換設定	タグ間の値の受け渡しを定義
I/O タイミング設定	入出力のタイミング調整
内部デバイス	KMX 内部だけで使う仮想 I/O を定義

「内部処理」モードで始める場合、**内部デバイス**に必要な仮想 I/O を作っておくと、PLC なしで信号のやり取りを再現できます。

ユニットを追加

「ユニット個別設定」を右クリックして「ユニット追加」を選びます。方法は2つです。

方法	内容
新規作成	ユニット名とユニットタイプを指定して1つ作る
DXF ファイルインポート	DXF から複数ユニットをまとめて作る

時短

ユニット名の入力欄を**ダブルクリック**すると複数行入力に切り替わり、**改行区切りで複数ユニットを一括登録**できます。

ユニットタイプは [§ 06](#) の 12 種類から選びます。迷ったら「タイムチャート」を選んでください（最も一般的）。

モデル設定

ユニット配下の「モデル設定」で、**3D モデルのどの部分を動かすか**を指定します。

項目	内容
グループ名	Prefab 内の親グループ名
モデル名	動かす対象のモデル名
子モデル設定	一緒に動く子モデル
衝突あり	チェックすると干渉判定の対象になる

注意

モデル名は Unity の Prefab 内のオブジェクト名と**一致している必要があります**。

［プロジェクト］ → ［Prefab 表示］（**Ctrl+D**）で構造を確認しながら入力してください。

動作設定

ユニットタイプに応じた設定ページが表示されます。ここでは最も一般的な**タイムチャート**を説明します。

項目	内容
モード	動作の種類（直線／回転など）
動作方向	軸の向き（＋／－）
オフセット	原点からのずれ（mm）
タグ	値の取得元タグ
サイクル倍率	1 サイクルに対する倍率
データ単位系	受け取る値の単位（倍率）
動作テーブル	時間と位置の対応表。ここが動きの本体
出力モード切替(直線⇒回転)	直線値を回転に変換する

動作テーブルが実際の動きを決めます。「時間 0ms のとき位置 0mm、時間 500ms のとき位置 100mm」のように登録すると、その通りに動きます。

設定ファイル出力

［プロジェクト］ → 〔設定ファイル出力〕（**Ctrl+E**）を実行します。Step 1・Step 3 で指定したフォルダに、30 種類以上の JSON ファイルが書き出されます（一覧は [付録 A](#)）。

12

KMX で確認

KMX を起動します。すでに起動している場合は **F5** で設定を再読み込みできます。

キー	動作
F5	全体を再読み込み（Prefab 含む）
Shift+F5	パラメータのみ再読み込み（速い）
Ctrl+F5	編集モードで再読み込み
F12	背景（部屋）の表示切替
Ctrl+Shift （押している間）	確認表示：経路ライン・周長・ワーク削除範囲
Ctrl+Shift+L	ログファイルを開く（不具合調査用）

06 ユニットタイプ一覧

ユニット追加時に選ぶ 12 種類です。

タイプ	用途	主な設定内容
モデルのみ	動かない。表示だけ	モデル設定のみ
タイムチャート	時間表に従って動く（最も一般的）	動作テーブル、モード、方向、オフセット
ロボット	多関節ロボット	J1～J6、位置／角度タグ、ヘッドオフセット
コンベア	搬送コンベア	速度、加速度、摩擦係数、方向
同期機構	他ユニットに同期して動く	同期元ユニット、開始／終了タグ
リニア	リニア搬送（周回）	ムーバー数、最小ピッチ、周長、ポイント
平面リニア	平面リニア搬送	シャトル数、ポイント、速度テーブル
スイッチ	押しボタン等	スイッチタイプ、動作モード、初期値
シグナルタワー	積層信号灯	赤／黄／緑／青／白のタグ
LED	LED 表示	LED タイプ、LED1～5 のタグ

タイプ	用途	主な設定内容
段ボール	製函・封函	製函モード、各フラップのモデル
型替え部品	品種切替で位置が変わる部品	品種ごとの位置・角度

07 データ通信設定の詳細

機番配下の「データ通信設定」にある 4 ページを個別に説明します。

7.1 使用可能タグリスト

その機番で使えるタグの一覧です。**編集するページではなく、確認用のページ**です。

システム設定で MMS 設定ファイル（.xml）を登録すると、PLC のタグ定義がここに読み込まれます。動作設定などでタグを指定するとき、ここに載っているタグが選択候補になります。

7.2 データ交換設定

あるタグの値を、別のタグへ写す設定です。1 行が「入力 → 出力」1 組に対応します。

列	内容
入力	値の取得元タグ
出力	値の書き込み先タグ
個数	まとめて写す個数（配列のとき）
備考	メモ

数値を直接入力することもできます（固定値の書き込み）。用途の例：

- PLC から受け取った値を、KMX 内部で使うタグへ移す
- あるユニットの出力を、別のユニットの入力につなぐ
- 初期値をタグへ書き込む

7.3 I/Oタイミグ設定

一定周期で ON / OFF を繰り返す信号を作る設定です。PLC がなくても、周期的な信号を発生させられます。

列	内容
名称	設定の名前
出力タグ	ON/OFF させるタグ
サイクル	繰り返し周期（msec）
ON タイミング	サイクル内で ON にする時刻（msec）
OFF タイミング	サイクル内で OFF にする時刻（msec）

例：サイクル 1000 / ON 0 / OFF 300 とすると、1 秒周期で最初の 300ms だけ ON になる信号になります。

7.4 内部デバイス（仮想 I/O）

KMX の中だけで使う、名前付きの信号を定義します。実機の PLC には存在しません。

列	内容
名称	信号の名前（機番内で一意）
配列数	1 なら単一、2 以上なら配列
ビット	チェックでビット、外すとワード
備考	メモ

定義した内部デバイスは、スイッチの出力タグとタイムチャートの開始タグ／終了タグで選択できるようになります。これにより「スイッチを押す → タイムチャートが動く」を、PLC なしで実現できます（§ 09 例3）。

`_IN[]` / `_OUT[]` との違い

内部処理モードでは `_in[0]`、`_out[3]` のような信号が自動採番されますが、これらは匿名で、狙って指定することができません。内部デバイスは自分で名前を付けて明示的に結線できる点が異なります。

配列にした場合のタグ名は、機番設定に従って `SIG[0]` 形式または `SIG0` 形式になります。実際に出力される名前は `InternalDevice.json` の `tags` で確認できます。

OFF 開始

タグ名の先頭に `-` を付けると **OFF 開始**（信号が OFF になったら動き出す）になります。内部デバイス・実 I/O どちらでも使える表記です。

08 拡張機能の詳細

ユニット配下の「拡張機能」にある 6 ページを説明します。**必要なものだけ追加**すればよく、すべて設定する必要はありません。

8.1 ワーク操作設定

ワーク（製品）の生成・削除・受け渡しをまとめて設定します。1 行が 1 つの操作です。

モード	動作
生成	ワークを作り出す
削除	ワークを消す
アタッチ	ワークをこのユニットに持たせる（掴む）
変換	ワークを別の種類に置き換える

主な列	内容
タグ	この操作を起こすトリガのタグ
ワーク名	対象のワーク（ワーク設定で定義したもの）
変換先	「変換」モードでの変換後のワーク
設計位置	チェックすると設計上の位置に配置
X/Y/Z (mm)、RX/RY/RZ (°)	位置と角度
範囲 (mm)	操作が効く範囲

主な列	内容
生存距離	ワークが消えるまでの距離
把持可能／接触可能／重力	ワークの振る舞い
周期生成／生成サイクル	一定周期で自動生成する
バケット番号	バケット式搬送での位置

8.2 センサ設定

ワークの通過を検出するセンサを作ります。

列	内容
生成	チェックするとセンサを作る
出力タグ	検出時に ON にするタグ
センサ幅 (mm)	検出範囲の幅
X / Y / Z (mm)、RX / RY / RZ (°)	取り付け位置と向き

出力タグに内部デバイスを指定すれば、センサの検出をそのまま他の動作のトリガにできます。

8.3 吸引設定

吸着パッドなどでワークを保持する設定です。

列	内容
吸引タグ	ON で吸着するタグ
吸引 OFF タグ	明示的に離すタグ
出力タグ	吸着中であることを知らせるタグ
位置固定 X/Y/Z、角度固定 X/Y/Z	吸着中に固定する軸
X / Y / Z (mm)、RX / RY / RZ (°)	吸着位置のオフセット

「位置固定」「角度固定」は、吸着した後にワークのどの成分を保持するか指定です。たとえばZだけ固定しないでおけば、吸着しながら高さは自由、という挙動になります。

8.4 物体形状設定

ユニットに**当たり判定用の形状（箱）**を持たせます。3D モデルの形が複雑な場合に、簡単な箱で干渉判定を代用できます。

列	内容
物体作成	チェックすると形状を作る
中心 X / Y / Z (mm)	箱の中心
サイズ X / Y / Z (mm)	箱の大きさ
自動設定	モデルの形から自動で箱を決める

「自動設定」を使えば手入力は不要です。モデル設定の「衝突あり」と併用できます。

8.5 機構拡張設定

複数のモデルが連動して動く機構を定義します。

項目	内容
機構タイプ	現在は LM ガイド
グループ名／モデル名	対象モデル（スライダ、従動軸）
子モデル設定	連動する子モデル

子モデルの種類	意味
通常	親と一緒に動く
回転中心	この点を中心に回転する
回転中心（固定）	回転中心だが位置は固定

LM ガイドのように「スライダが動くと従動軸が追従する」構造を表現するために使います。

8.6 バケット設定

バケット（ポケット）が並んで循環する搬送機構を定義します。

項目	内容
ベルトグループ名／ベルトモデル名	循環するベルト側のモデル
バケット数	並べる個数
バケットピッチ（mm）	間隔
オフセット	開始位置のずらし量
バケット非表示	バケット自体を表示しない
常に上向き（吊り下げ式）	循環しても姿勢を保つ
経路名	経路設定で定義した経路名。指定するとベルトモデルより優先

経路の決め方	内容
ベルトモデルから自動生成	ベルトの外形から経路を作る
経路設定を参照（推奨）	機番配下の「経路設定」で定義した経路を「経路名」で指定

経路設定では、経路を **スプロケット**（円弧）と **経由点**（直線の通過点）の組み合わせで定義します。

確認方法

KMX 実行中に

Ctrl+Shift

 を押している間、経路ラインと周長が画面に表示されます。意図した経路になっているかはこれで確認できます。

09 設定例

例 1：タイムチャートで 1 軸を動かす（PLC なし）

やりたいこと：X 軸のスライダを、1 サイクル 2000ms のうち 0→500ms で 0mm から 300mm まで動かす。

手順	設定内容
1	PostgreSQL 設定の機能で「内部処理」を選択
2	タイムチャート設定の 1 サイクル時間 = 2000 (msec)
3	ユニット追加：名前 <code>スライダX</code> 、タイプ タイムチャート
4	モデル設定：モデル名に Prefab 内の該当オブジェクト名を入力
5	動作設定：モード＝直線、動作方向＝＋、動作テーブルに下表を登録
6	<code>Ctrl+E</code> で出力 → KMX で <code>F5</code>

動作テーブル

時間 (ms)	位置 (mm)
0	0
500	300
2000	300

これで PLC がなくてもスライダが動きます。

例 2：PLC の実データで動かす（直接通信）

やりたいこと：三菱 PLC の `D12128` から読んだ機構位置で軸を動かす。

手順	設定内容
1	PostgreSQL 設定の機能で「直接通信」を選択
2	通信設定：プロトコル＝MC プロトコル、IP＝ <code>192.168.3.36</code> 、ポート＝ <code>12290</code>
3	データ通信設定でタグを登録（レジスタ種別 <code>D</code> 、レジスタ番号 <code>12128</code> ）
4	動作設定の「タグ」に、登録したタグを指定
5	「データ単位系」で PLC の値の倍率を指定

実際に出力される JSON（抜粋）

`Postgres.json` — 接続設定

```
[{
  "No": 1, "Cycle": 10,
  "Server": "192.168.3.210", "Port": 5432,
  "Database": "kcp_db", "User": "kcp_user",
  "ClientMode": 0, "DirectMode": 1,
  "directDatas": [{
    "mechId": "R0230",
    "protocol": 0,
    "IpAddress": "192.168.3.36",
    "PortNo": 12290,
    "tags": [{
      "Name": "機構位置",
      "RegisterType": "D",
      "RegisterNo": 12128,
      "DataCount": 31,
      "DataTag": "d_mech_pos",
      "IsWrite": false
    }]
  }]
}]
```

ActionInfo.json — 動作設定

```
[{
  "mechId": "R0230",
  "name": "シート取出・箱立X軸",
  "mode": 2,
  "axis": 0,
  "delay": 100,
  "tag": "d_mech_pos[0].x",
  "rate": 1000000,
  "dir": -1,
  "offset": 150
}]
```

フィールド	意味
tag	値を読むタグ。 <code>d_mech_pos[0].x</code> は機構位置の 0 番目の X
rate	データ単位系。 <code>1000000</code> は PLC 値が 1/1,000,000 単位であることを示す
dir	動作方向。 <code>-1</code> は逆向き

フィールド	意味
offset	原点オフセット (mm)
axis	動かす軸 (0=X, 1=Y, 2=Z)

例 3：画面のスイッチで動かす（内部デバイス）

やりたいこと：画面に置いたスイッチを押したら、タイムチャートの動作が始まるようにする。PLC は使わない。

内部デバイスを「スイッチ」と「タイムチャート」の間の配線として使います。



手順	設定場所	設定内容
1	データ通信設定 → 内部デバイス	名称 SW_TIMECHART_TRG、配列数 1、ビット✓
2	ユニット追加	名前 運転ボタン、タイプ スイッチ
3	スイッチの動作設定	モード=タグ出力、出力タグ= SW_TIMECHART_TRG、種類=オルタネート（保持）
4	動かしたいユニットの動作設定	該当動作の開始タグに SW_TIMECHART_TRG を指定
5	—	Ctrl+E で出力 → KMX で F5

出力される JSON

```
// InternalDevice.json
[{ "mechId":"M001", "name":"SW_TIMECHART_TRG",
  "count":1, "isBit":true, "tags":["SW_TIMECHART_TRG"]} ]

// SwitchInfo.json
[{ "mechId":"M001", "name":"運転ボタン",
  "mode":0, "alternate":true, "tag":"SW_TIMECHART_TRG" }]

// ActionInfo.json (該当アクション)
{ "trg":0, "target":100.0, "start":"SW_TIMECHART_TRG", "time":0.5 }
```

3つのファイルで **同じ文字列** `SW_TIMECHART_TRG` が使われている点がポイントです。KMXはこの文字列一致で信号をつなぎます。

応用

スイッチを押している間だけ動かしたい場合は、種類を「モーメンタリ（一時）」にします。逆に「信号がOFFになったら動く」ようにしたい場合は、開始タグを `-SW_TIMECHART_TRG` のように `-` 付きで指定します。

10 覚えておきたい約束事

単位

場所	単位
KMXTool の入力欄	mm
出力される JSON	基本は m （KMXTool が 1/1000 して出力）
例外： <code>PathInfo.json</code> の <code>loopLength</code>	mm のまま出力（KMX 側で 1/1000 する）

覚え方

KMXTool には **mm** で入力する、とだけ覚えておけば大丈夫です。

変更が反映される流れ



最頻出のミス

Ctrl+E (設定ファイル出力) の忘れが、圧倒的に多いミスです。「反映されない」と思ったらまずここを確認してください。

11 よくあるつまずき

「下記パスは不正です」と表示される

出力先フォルダの指定が間違っています。§ 05 Step 1 の 3 パターンのいずれかに合わせてください。最も確実なのは **KMX.exe** があるフォルダを指定することです。

KMX に変更が反映されない

確認	対処
設定ファイル出力をしたか	Ctrl+E を実行
KMX で再読込したか	F5 を押す
出力先が KMX の場所と一致しているか	基本設定の Unity データパスを確認
出力先を 2 か所指定していないか	システム設定の保存フォルダパスも確認。片方だけ更新されている可能性

モデルが表示されない・動かない

確認	対処
モデル名が Prefab と一致しているか	<code>Ctrl+D</code> （Prefab 表示）で正しい名前を確認
非表示モデル設定に入っていないか	機番配下の「非表示モデル設定」を確認
ユニットが無効になっていないか	ユニットの有効／無効、品種別の有効／無効を確認

動く量がおかしい（1000倍・1/1000）

データ単位系（`rate`）の設定を確認してください。PLC 値の単位と KMX 側が期待する単位が合っていないと、桁がずれます。§ 09 の `rate: 1000000` が例です。あわせて、入力を `mm` で行っているかも確認してください。

原因が分からない

KMX 上で `Ctrl+Shift+L` を押すとログファイルが開きます。読み込みエラーやタグ未検出などが記録されています。

12 用語集

用語	意味
機番	機械 1 台を識別する ID。例 <code>R0230</code>
ユニット	動くもの 1 つ分の設定単位
タグ	PLC の値につけた名前。 <code>d_mech_pos</code> など
デバイス	PLC 上のアドレス。 <code>D12128</code> など
タイムチャート	時間と位置の対応表。動きの定義
動作テーブル	タイムチャートの中身（時間・値の並び）
サイクル	機械が 1 周する動作。その時間が 1 サイクル時間
ワーク	機械が扱う製品・部品
品種	製品の種類。品種ごとに位置や有効／無効を変えられる

用語	意味
Prefab	Unity の 3D モデル部品
内部処理	PLC につながず KMX 内部だけで動かすモード
直接通信	PLC と直接通信するモード
MMS 設定ファイル	PLC のタグ定義が入った XML

付録 A 出力される設定ファイル一覧

設定ファイル出力（ `Ctrl+E` ）で書き出されるファイルです。すべて `StreamingAssets\Datas` に出力されます。

ファイル	内容
<code>Postgres.json</code>	接続設定・通信設定・タグ定義
<code>UnitInfo.json</code>	ユニット構成
<code>ActionInfo.json</code>	動作設定
<code>ActionTableInfo.json</code>	動作テーブル（時間・値の並び）
<code>DataExchangeInfo.json</code>	データ交換設定
<code>InternalDevice.json</code>	内部デバイス（仮想 I/O）
<code>InnerProcess.json</code>	内部処理設定
<code>HiddenUnitInfo.json</code>	非表示ユニット
<code>WorkModelInfo.json</code>	ワーク定義
<code>WorkCreateInfo.json</code>	ワーク生成
<code>WorkDeleteInfo.json</code>	ワーク削除
<code>WorkTransferInfo.json</code>	ワーク受渡
<code>RobotInfo.json</code>	ロボット設定
<code>ConveyerInfo.json</code>	コンベア設定

ファイル	内容
LinearInfo.json	リニア設定
PlanarMotorInfo.json	平面リニア設定
ChuckUnitInfo.json	チャックユニット
SensorInfo.json	センサ設定
SuctionInfo.json	吸引設定
ShapeInfo.json	物体形状設定
ExMechInfo.json	機構拡張設定
BucketInfo.json	バケット設定
PathInfo.json	経路設定
SwitchInfo.json	スイッチ設定
SignalTowerInfo.json	シグナルタワー設定
LedInfo.json	LED 設定
CardboardInfo.json	段ボール設定
ChangeOverInfo.json	型替え部品設定
PrefabInfo.json	Prefab 情報
UseDeviceList.json	使用デバイス一覧
TimeChartDataList.json	タイムチャートデータ一覧
DebugInfo.json	デバッグ情報
BuildConfig.json	ビルド設定

一部のファイルは、該当する設定がない場合は中身が空（`[]`）で出力されます。異常ではありません。

KMXTool 設定マニュアル ソース：`docs/KMXTool設定マニュアル.md`